

JavaScript Array Method - splice()

The `splice()` method is one of the most powerful array methods in JavaScript. It is used to **add, remove, or replace elements** in an array.

Unlike `slice()`, the `splice()` method **modifies the original array**.

Table of Contents

- What is splice()?
- Why Use splice()?
- Syntax
- Parameters
- Return Value
- How splice() Works
- Basic Examples
- Multiple Examples
- Real Project Examples
- Best Practices
- Common Mistakes
- Interview Questions
- Summary Table

What is splice()?

Definition

The `splice()` method changes the contents of an array by:

- Removing elements
- Adding new elements
- Replacing existing elements

It modifies the **original array**.

Why Use splice()?

We use `splice()` when we need to:

- Remove items
- Insert new items
- Replace existing items
- Update array data
- Manage dynamic lists

Syntax

```
array.splice(start, deleteCount);  
  
array.splice(start, deleteCount, item1);  
  
array.splice(start, deleteCount, item1, item2, ...);
```

Parameters

Parameter	Description
start	Index where the operation begins
deleteCount	Number of elements to remove
item1, item2...	New elements to insert (Optional)

Return Value

Returns an array containing the **removed elements**.

The original array is modified.

How splice() Works

Original Array

```
["Apple", "Mango", "Orange", "Banana"]
```

↓

```
splice(1,2)
```

↓

Removed

```
["Mango", "Orange"]
```

↓

Remaining Array

```
["Apple", "Banana"]
```

Example 1 (Remove Elements)

```
const fruits = [  
  "Apple",  
  "Mango",  
  "Orange",  
  "Banana"  
];  
  
fruits.splice(1,2);
```

```
console.log(fruits);
```

Output

```
["Apple", "Banana"]
```

Explanation

- Start from index **1**
- Remove **2** elements
- Original array changes

Example 2 (Store Removed Elements)

```
const numbers = [  
  10,  
  20,  
  30,  
  40  
];  
  
const removed = numbers.splice(1,2);  
  
console.log(removed);  
  
console.log(numbers);
```

Output

```
[20,30]
```

```
[10,40]
```

Example 3 (Add Elements)

```
const colors = [  
  "Red",  
  "Blue"  
];  
  
colors.splice(1,0,"Green");  
  
console.log(colors);
```

Output

```
["Red","Green","Blue"]
```

Explanation

- Start at index **1**
- Remove **0** elements
- Insert **"Green"**

Example 4 (Replace Elements)

```
const fruits = [
```

```
    "Apple",  
  
    "Mango",  
  
    "Orange"  
];  
  
fruits.splice(1,1,"Banana");  
  
console.log(fruits);
```

Output

```
["Apple","Banana","Orange"]
```

Example 5 (Replace Multiple Elements)

```
const numbers = [  
  
    10,  
  
    20,  
  
    30,  
  
    40  
];  
  
numbers.splice(1,2,100,200);  
  
console.log(numbers);
```

Output

```
[10,100,200,40]
```

Example 6 (Insert Multiple Elements)

```
const letters = [  
  "A",  
  "D"  
];  
  
letters.splice(  
  1,  
  0,  
  "B",  
  "C"  
);  
  
console.log(letters);
```

Output

```
["A","B","C","D"]
```

Example 7 (Remove All Elements After an Index)

```
const data = [  
  1,
```

```
    2,  
    3,  
    4,  
    5  
  ];  
  
  data.splice(2);  
  
  console.log(data);
```

Output

```
[1,2]
```

Explanation

Everything from index **2** onward is removed.

Example 8 (Negative Index)

```
const numbers = [  
  10,  
  20,  
  30,  
  40  
];  
  
numbers.splice(-2,1);  
  
console.log(numbers);
```

Output

```
[10,20,40]
```

Real Project Example 1

Remove Product from Shopping Cart

```
const cart = [  
  "Laptop",  
  "Mouse",  
  "Keyboard"  
];  
  
const index = cart.indexOf("Mouse");  
  
cart.splice(index,1);
```

```
console.log(cart);
```

Output

```
["Laptop", "Keyboard"]
```

Real Project Example 2

Delete Student

```
const students = [  
  "Rahim",  
  "Karim",  
  "Rokon"  
];  
  
students.splice(1,1);  
  
console.log(students);
```

Output

```
["Rahim", "Rokon"]
```

Real Project Example 3

Insert VIP Customer

```
const queue = [  
  "Customer 1",  
  "Customer 2"  
];  
  
queue.splice(0,0,"VIP Customer");  
  
console.log(queue);
```

Output

```
[  
  "VIP Customer",  
  "Customer 1",  
  "Customer 2"  
]
```

Real Project Example 4

Replace Notification

```
const notifications = [  
  "Old Message"  
];  
  
notifications.splice(  
  0,  
  1,
```

```
    "New Message"  
  );  
  
  console.log(notifications);
```

Output

```
["New Message"]
```

Real Project Example 5

Update Product List

```
const products = [  
  "Laptop",  
  "Mouse",  
  "Keyboard"  
];  
  
products.splice(  
  2,  
  1,  
  "Monitor"  
);  
  
console.log(products);
```

Output

```
[  
  "Laptop",  
  "Mouse",  
  "Monitor"  
]
```

Best Practices

Use `splice()` when you need to modify the original array.

Store removed elements if needed.

```
const removed = array.splice(1,2);
```

Use `indexOf()` before `splice()` when removing a specific value.

```
const index = array.indexOf(value);  
  
if(index !== -1){  
  array.splice(index,1);  
}
```

Use meaningful variable names.

```
removedItems  
  
updatedProducts  
  
deletedUsers
```

Common Mistakes

Mistake 1

Confusing `splice()` with `slice()` .

```
slice()
```

Creates a copy.

```
splice()
```

Changes the original array.

Mistake 2

Forgetting that `splice()` modifies the original array.

```
const arr = [1,2,3];  
  
arr.splice(1,1);  
  
console.log(arr);
```

Output

```
[1,3]
```

Mistake 3

Ignoring the return value.

```
const removed = arr.splice(1,2);
```

The returned array contains the removed elements.

Mistake 4

Using an invalid index.

```
const arr = [1,2];  
  
arr.splice(10,1);
```

Output

```
[]
```

Nothing is removed.

Interview Questions

What does `splice()` do?

It adds, removes, or replaces elements in an array.

Does `splice()` modify the original array?

Yes.

What does `splice()` return?

An array containing the removed elements.

Which parameter specifies where to start?

```
start
```

Which parameter specifies how many elements to remove?

```
deleteCount
```

Can `splice()` insert elements?

Yes.

Can `splice()` replace elements?

Yes.

Which method copies an array without modifying it?

```
slice()
```

Summary Table

Feature	Description
Method	<code>splice()</code>
Purpose	Add, remove, or replace elements
Parameters	start, deleteCount, items
Returns	Removed elements

Feature	Description
Changes Original Array	Yes
Supports Negative Index	Yes
Common Uses	CRUD Operations, Shopping Cart, Todo List

Quick Comparison

Method	Purpose	Returns	Changes Original Array
<code>slice()</code>	Copy elements	New Array	No
<code>splice()</code>	Add / Remove / Replace	Removed Elements	Yes

Final Notes

The `splice()` method is one of the most important array methods in JavaScript because it allows you to **insert, delete, and replace elements** in a single method.

It is widely used in:

- Shopping Cart Applications
- Todo List Applications
- CRUD Operations
- Queue Management
- Dynamic Tables
- React State Updates (using copied arrays)

Mastering `splice()` is essential before learning advanced array methods like `map()` , `filter()` , `find()` , and `reduce()` .